String

String is a sequence of characters.
A character array is a string if it ends with a null character ('\0').

The ASCII value of null character '\0' = 0

Initialization of string :-

Char str[] = {'B', 'H', 'O', 'P', 'A', 'L', '\0'};

Char str[] = {'B', 'H', 'O', 'P', 'A', 'L'};

Char str[] = "BHOPAL";

Char str[10] = {'B', 'H', 'O', 'P', 'A', 'L'};

	0	1	2	3	4	5	6	7	8	9
str	B	H	O	P	A	L	\0	\0	\0	\0

	0	1	2	3	4	5	6	7	8	9
str	66	72	79	80	65	76	0	0	0	0

In an array zero (0) is stored as integer not character

Integer 0
Character '0'

format specifier

%c → To print character value

%d → To print integer value
or To print ASCII value

```
#include<stdio.h>
int main()
{
    char arr[] = "Bhopal";
    int i=0;
    while(i<6)  → Here we need to count our char element
    {
        printf("%c", arr[i]);
        i++;
    }
}
```

OUTPUT :-

Bhopal

```
#include<stdio.h>
int main()
{
    char arr[] = "Bhopal10";
    int i=0;
    while(arr[i]!=10)
    {
        printf("%c", arr[i]);
    }
    return 0;
}
```

10 is optional to write, the compiler by-default add it

→ Here it is no need to count no of element in Character

OUTPUT :-

Bhopal

```
#include<stdio.h>
int main()
{
    char str[5]="hello";
    int i=0;
    while ( str[i] != '\0')
    {
        printf("%c ",str[i]);
        i++;
    }
}
```

→ The actual size of char str is 6 and we defined it 5

OUTPUT :-

It will not give the required output

hello ♣

```
#include<stdio.h>
int main()
{
    char str[] = {'M','i','t','t','h','u'};
    int i=0;
    while (str[i] != '\0')
    {
        printf("%c ",str[i]);
        i++;
    }
}
```

↳ In this way of initialization the c/p doesn't by-default add \0.

OUTPUT :-

m i t t h u ♠ { Not accurate }

Accessing individual characters :

```
Char str[] = "Physics Wallah";
```

(a) `printf("%c", str[5]);` // c
(b) `printf("%d", str[9]);` // 97

Modifying individual characters :

```
Char str[] = "Physics Wallah";
```

(a) `str[0] = 'M';`

Mhysics Wallah

(b) `str[1] = 97;`

Paysics Wallah

format specifies :-

%c - To print single character

%s - To print whole string at once

```
#include<stdio.h>
int main()
{
    char str[]={'B','H','O','P','A','L'};
    printf("%s",str);
}
```

↳ only array name

OUTPUT

BHOPAL

Calculating length of the string

```
#include<stdio.h>
int main()
{
    char str[]={'B','H','O','P','A','L'};
    int i;
    for(i=0; str[i]; i++);
    printf("Length is %d ",i);
}
```

→ It means there is no body of for loop

Taking Input from user :

%s → To read or To print string

- By using scanf()

```
char S1[10];
printf("Enter name :");
scanf("%s", S1);
```

Here we doesn't use for loop because we doesn't need to specify the address of each character.

Here we not need to use & (address of) because we already passing address by using S1.

```
printf("%s", S1);
```

- scanf is not capable to input multiword string.
- It consider space, tab, new line characters as terminating point.
- It by default use '\0' whenever it face space, tab or new line characters.

gets() → only used to input string

```
#include<stdio.h>
int main
{
    char name[30];
    printf("Enter name :");
    gets(name);
    printf("%s",name);
}
```

one time only one string can input

name → Array name

O/P:

Enter name : Gaurav Kandpal
Gaurav Kandpal

It is risky to ~~the~~ use scanf() and gets() function due to buffer overflow.
They doesn't check buffer size while taking input

fgets

fgets(array name , array size , from where we are taking input)

fgets(array name, array size , stdin)

↳ when we take input from keyword

```
#include<stdio.h>
int main()
{
    char str[30];
    printf("Enter your name : ");
    fgets (str , 30, stdin);      fgets() reads 29 characters from
    printf ("%s", str);           starting + 1 for '\0'
}
```

O/P

Enter your name : Mysirg Education Services Private limited
Mysirg Education Services Pri

We can read/store specific value using scanf :-

```
#include<stdio.h>
int main()
{
    char name[10];
    printf("Enter your name :");
    scanf("%7s", name);          scanf will store only first
    printf("%s", name);          7 characters
}
```

Output :-

Enter your name : Kandpal gaurav
Kandpal

```
#include<stdio.h>
int main()
{
    char name[30];
    printf("Enter your name :");
    scanf("%s", name);
    printf("%.5s", name);
    printf("\n%10.5s", name);    # 10 block filled with 5
}
```

Output :-

Enter your name : GauravKandpal
Gaura
     Gaura

puts() function {pre-defined}

```
#include<stdio.h>
int main()
{
    char name[30];
    printf("Enter your name :");
    gets(name);
    puts(name);
}
```

Output :-
Enter your name : Gaurav Kandpal
Gaurav Kandpal

```
#include<stdio.h>
int main()
{
    Char name1[30];
    printf("Enter your name :");
    gets(name1);
    puts(name1);
    puts(name1);
}
```

Output :-
Enter your name : Gaurav Kandpal
Gaurav Kandpal
Gaurav Kandpal

→ puts() will add automatically new line operator

Memory concept :

```
Char str[10] = "Bhopal";
int i;
for(i=0 ; str[i]; i++);   # printf doesn't know what is after B because
printf("%c", str[i]);  → we are just passing character B not the string address
```

↳ Here (i=0) we are passing character 'B' not address so printf is not able to print the whole string.
So to print whole string we are using for loop

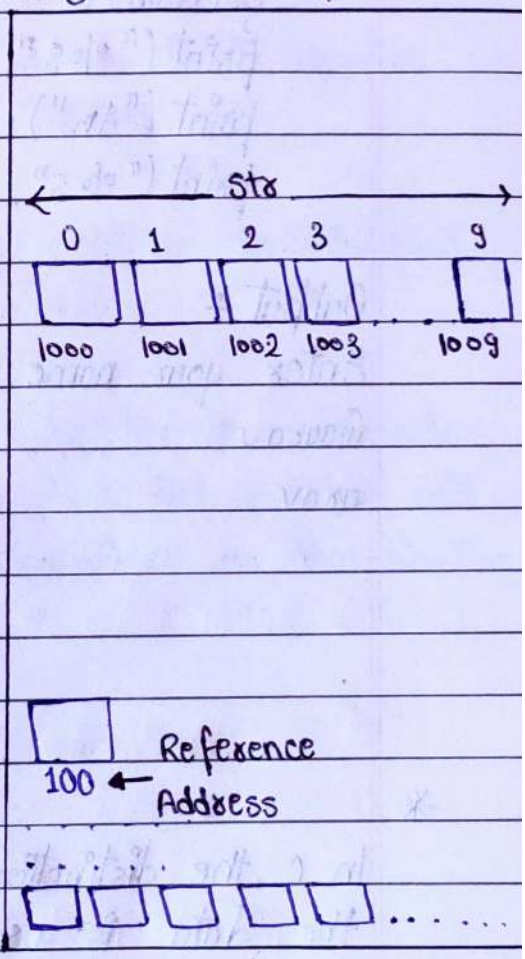
```
printf("%s", str);
```

↳ Here we are passing the base address

Array name always give you base address

str ≈ &str[0] ≈ 1000

Program's memory



Position number of bit is known as address or Reference

```
#include <stdio.h>
int main()
{
    char str[30];
    printf("Enter your name = ");
    gets(str);
    print("%s", &str);  → It is technically incorrect
    print("\n");
    print("%s", &str[2]);
                  └→ It is start printing from index 2.
```

Output :-

```
Enter your name : Gaurav
Gaurav
urav
```

*

In C, the distinction b/w an array and a string is based on how the data is used. The compiler does not inherently understand whether an array is being used as a string or not. It's up to the programmer to ensure, that it an array or a string.

String library functions :-

#include<string.h>

1. Strlen() → To find length of a string

Declaration : size_t strlen(char *string)
unsigned int ↳ Pointer to a character string
↳ return type

strlen() accepts a single argument, which is pointer to the first character of the string.

* When we declare a variable as a pointer to a character using char*, we're essentially saying that this variable will hold the memory address of a character, or the starting address of a string

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[20]="hello";
    printf("length of string = %d",strlen(str));
}
```

Here we use str instead of char *str because in C, when you declare an array or string such as char str[20]; the array or string name str can be used as a pointer to the first element of the array or string. This is a special case in C, where the array name can act as a pointer to its first element.

without strlen()

```
#include<stdio.h>
int main
{
    int i;
    int count = 0;
    Char str[30] = "Mahadev";
    while ( str[i] != '\0')
    {
        count++;
        i++;
    }
    printf ("length of the string = %d", count);
}
```

strcat () → concatenation of two strings

* The destination string must have enough size to append the source string But the string library function doesn't check the buffer size and append the string which can create the problem of buffer overflow if the destination string doesn't have enough size to append the source string

```
#include<stdio.h>
# include<string.h>
int main()
{
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";
    strcat(str1, str2);
    printf("%s", str1);
}
```

O/P :

GauravKandpal

strncat() → it concatenates only a portion of a string to another string

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";
    strcat(str2, str1, 1);
    printf("%s", str2);
}
```

O/P :

KandpalG

We can also write strcat or strncat program in this way also :

```
#include<stdio.h>
# include<string.h>
int main()
{
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";
    strcat(str1, str2);
    printf("%s", str1);
    strcat(str2, "Gaurav");
    printf("\n%s", str2);
}
```

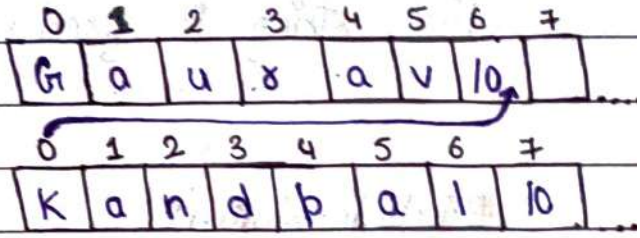
→ Here we have to write pointer to a character string but we have written string literal because in C string literals are implemented as arrays of character and their name can be used as a pointer to the first character. String literal in C is automatically treated as a pointer to the first character of that string.

Output :

GauravKandpal
KandpalGaurav

without strcat ( )

```
#include<stdio.h>
int main()
{
    int len1, len2;
    char str1[30] = "Gaurav";
    char str2[30] = "Kandpal";
    len1 = strlen(str1);
    len2 = strlen(str2);
    for(i=0 ; i<=len2 ; i++)
    {
        str1[len1+i] = str2[i];
    }
    printf("%s", str1);
}
```



Output :-

GauravKandpal

3. Strcmp() → for comparison of two string

strings are equal = 0
first string is less than the second = -ve value
first string is greater than the second = +ve value

```
#include<stdio.h>
#Include<string.h>
int main()
{
    Char str1[30] = "Gaurav";
    Char str2[30] = "gaurav";
    strcmp(str1, str2);
    printf("%d", str1);
    strcmp(str2, str1)
    printf("\n%d", str2);
}
```

→ The name of string/array without an index refers to the base address of the array.

we have to the result of strcmp in a variable and print that variable

Output :

6422274 → Not correct because we are directly printing the address of the first character in each string.
6422244

```c
#include<stdio.h>
#include<string.h>
int main()
{
    int result;
    char str1[30] = "Gaurav";
    char str2[30] = "gaurav";
    result = strcmp(str1, str2);
    printf("%d", result);
    result = strcmp(str2, str1);
    printf("%d", result);
}
```

G = 71 , g = 103

Output :-

-1 (any -ve value)

1 (any +ve value)

without strcmp()

```c
#include<stdio.h>
int main()
{
    int flag = 0;
    char s1[30] = "Hello";
    char s2[30] = "Hillo";
    for(int i=0; s1[i]!='\0' || s2[i]!='\0'; i++)
    {
        if(s1[i] != s2[i])
        {
            flag = 1;
            break;
        }
    }
```

```
    if(flag==1)
    printf("String are different");
    else
    printf("String are same");
}
```

4. strcpy() ⟶ used for copying one string to another string

```
#include<stdio.h>
#include<string.h>
int main()
{
        char str1[30]="Gaurav";
        char str2[30]="Kandpal";
        strcpy(str1, str2);
        printf("%s",str1);
}
```

Output :

Kandpal

Function call by passing string :-

```
#include<stdio.h>
void strcmp(char *str1, char *str2);
int main()
{
    char str1[30];
    char str2[30];
    printf("Enter a string str1 :");
    fgets(str1,30,stdin);
    printf("Enter a string str2 :");
    fgets(str2,30,stdin);
    strcmp(str1,str2);
    return 0;
}
void strcmp(char *str1, char *str2)
{
    int flag=0;
    for(int i=0; str1[i]!='\0' || str2[i]!='\0'; i++)
    {
        if(str1[i]!=str2[i])
        {
            flag=1;
            break;
        }
    }
    if(flag==1)
    printf("string are not same");
    else
    printf("string are same");
}
```

We can write them also as (char str1[], char str2[]) but due to conventional & common practice.

The main thing we must know that Array/string names are pointer to its first element

And same in array we can write them also as (int *a, int *b) instead of (int a[], int b[])

But this notation makes it clear that the function expects an array of integers.

Handling of Multiple string

size of each string

```
Char str [6][8] = { "Gaurav", "Suraj", "Chandu", "Kishor"
                    "Kannu", "Deepu" };
```

no. of string

	65456	65457	65458	65459	65460	65461	65462
65454	G	a	u	r	a	v	\0
65463	S	u	r	a	j	\0	
	C	h	a	n	d	u	\0
	K	i	s	h	o	r	\0
	K	a	n	n	u	\0	
	D	e	e	p	u	\0	

str[0] → represents 0th string, points to 0th character of 0th string

str[1] → represents 1th string, points to 0th character of 1th string

str[i][j] → represents jth character in ith string

String :

We can swap string by two method :

1. By one by one swapping each character

```
ch = s1[i];
s1[i] = s2[i];
s2[i] = ch;
```

2. By using string standard library function - strcpy

```
char temp[20];
strcpy(char temp
```

Some non-standard string library function :

1. strrev()

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[30] = "Gaurav Kandpal";
    printf("string after reverse : %s", strrev(str));
}
```

Output :
string after reverse : lapdnaK varuaG

without strrev()

```
#include<stdio.h>
int main()
{
    char ch;
    char str[30] = "Gaurav Kandpal";
    int l = strlen(str);
    for(int i=0; i<l/2; i++)
    {
        ch = str[i];
        str[i] = str[l-1-i];
        str[l-1-i] = ch;
    }
    printf("string after reversing : %s", str);
}
```

Output :
lapdnak varuaG

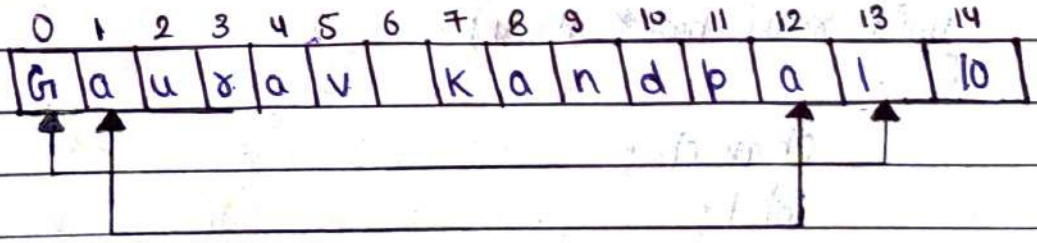
i

Ø1

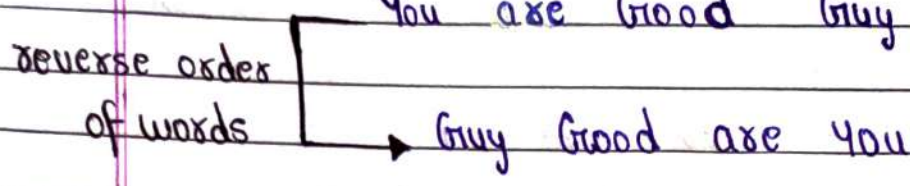
L = 14

i < 14/2 = 7

i=0	i=1	i=6
ch = G	ch = a	ch = ' '
str[0] = str[13] (l)	str[1] = str[12] (a)	str[6] = str[7] (K)
str[13] = G	str[12] = a	str[7] = ' '



To reverse order of words




```
#include<stdio.h>
#include<string.h>
void rev(char *strev);
int main()
{
    char str[40];
    printf("Enter a string:");
    gets(str);
    puts(str);
    rev(str);
    printf("string after reversing :%s",str);
    return 0;
}
void rev(char *strev)
{
    char ch;
    int l;
    l=strlen(strev);
    for(int i=0;i<l/2;i++)
    {
        ch=strev[i];
        strev[i]=strev[l-1-i];
        strev[l-1-i]=ch;
    }
```

```
int i=0 ; j=0 ; k=0;
for(i=0; strev[i]; i++)
 {
    if (strev[i] == ' ' || strev[i+1] == '\0')
    {
        for(k=(strev[i+1] == '\0' ? i: i-1); j<k; j++, k--)
        {
            ch= strev[j];
            strev[j] = strev[k];
            strev[k] =ch;
        }
        j=i+1;
    }
 }
}
```

2. strupr ( )
3. strlwr ( )

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[] = "Gaurav";
    strlwr(str);
    printf("string in lower case:%s", str);
    strupr(str);
    printf("%string in upper case:%s", str);
}
```

without this

```
#include<stdio.h>
void strupr(char *str)
void strlwr(char *str)
int main()
{
    char str[50];
    printf("Enter a string:");
    fgets(str, 50, stdin);
    strupr(str);
    printf("string in upper case:%s"str);
    strlwr(str);
    printf("string in lower case:%s"str);
    return 0;
}
```

```
void strupr (char * str)
{
    int i=0;
    while (str[i] != '\0')
    {
        if ( str[i] >= 'a' && str[i] <= 'z')
        {
            str[i] = str[i] - 32 ;
        }
        i++ ;
    }
}

void strlwr (char *str)
{
    int i=0
    while ( str[i] != '\0')
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
        {
            str[i] = str[i] +32;
        }
        i++;
    }
}
```